BIDIRECTIONAL SEARCH

```
Algorithm: BidirectionalSearch (start, goal)
    Queue forward_queue, backward_queue
    Map forward_visited, backward_visited
    Map forward_parent, backward_parent

    forward_queue.push(start)
    forward_visited[start] = true
    forward_parent[start] = null

    backward_queue.push(goal)
    backward_visited[goal] = true
    backward_parent[goal] = null

    intersecting_node = null
    while forward_queue is not empty AND backward_queue
                                        is not empty:

        if forward_queue is not empty:
            current_f = forward_queue.pop()

            if backward_visited[current_f] is true:
                intersecting_node = current_f
                break
            for each neighbor in GetNeighbors(current_f):
                if forward_visited[neighbor] is not true:
                    forward_visited[neighbor] = true
                    forward_parent[neighbor] = current_f
                    forward_queue.push(neighbor)

        if backward_queue is not empty:
            current_b = backward_queue.pop()
            if forward_visited[current_b] is true:
                intersecting_node = current_f
                break
```

```
    for each neighbor in GetNeighbors(current_b):
        if backward_visited[neighbor] is not true:
            backward_visited[neighbor] = true
            backward_parent[neighbor] = current_b
            backward_queue.push(neighbor)

if intersecting_node is not null:
    return ReconstructPath(intersecting_node, forward_parent,
                           backward_parent)

else:
    return "No path found"

Function ReconstructPath(node, forward_parent, backward_parent):
    path = []
    curr = node
    while curr is not null:
        path.prepend(curr)
        curr = forward_parent[curr]

    curr = backward_parent[node]
    while curr is not null:
        path.append(curr)
        curr = backward_parent[curr]

    return path
```

Output: Path found: A → B → C → D → Goal

